

Uma DSL Híbrida para Jogos RPG com Suporte a Eventos de Gameplay

1st Gabriel Franklin Martins Lazzarini Miranda
Engenharia da Computação
Centro Universitário - Fundação Hermínio Ometto
Araras, Brasil
gabrielmmlm@alunos.fho.edu.br

2nd Lucas Fortolan Sampaio
Engenharia da Computação
Centro Universitário - Fundação Hermínio Ometto
Araras, Brasil
lucas.fortolan@alunos.fho.edu.br

Abstract—The development of Role-Playing Games (RPGs) involves the construction of complex interactive narratives, combining structural representation of story elements with dynamic execution mechanics. However, using general-purpose programming languages to implement these systems frequently results in high coupling between control logic and narrative modeling, hindering code maintenance, readability, and evolution. To address this challenge, this paper presents the design, implementation, and evaluation of a hybrid Domain-Specific Language (DSL) tailored for modeling narrative RPGs, culminating in its final version, V4. The developed system incorporates classic compiler construction techniques, including lexical, syntactic, and preventive static semantic analysis based on a context-free grammar processed via LALR(1) algorithms using the PLY (Python Lex-Yacc) library. Program execution is carried out through an interpreted Abstract Syntax Tree (AST) in a Python environment. Version V4 introduces significant structural and logic advancements over prior iterations: the integration of an explicit conditional flow control structure (if-then) backed by relational operators, granular variable state mutations (set, add, and remove), explicit source code documentation via comment tokens, and an active runtime crash-prevention routine governed by a Conditional Game Over monitoring loop. To ensure high software engineering standards, the development environment was fully refactored, segregating the cells to mirror the classic pipeline of traditional compilers. The language was validated through the compilation and execution of a large-scale experimental script titled "Horizonte Rubro e o Mar Invertido". The experimental results demonstrated that the DSL successfully abstracts low-level programming complexities into intuitive, high-level narrative primitives, while the preventive static semantic analyzer effectively eliminates catastrophic runtime failures caused by identifier collisions, type mismatches, or orphan scene graphs, delivering a highly decoupled, secure, and performant architecture for interactive storytelling.

Index Terms—Linguagem Específica de Domínio, RPG, Jogos Narrativos, Compiladores, Python, Desenvolvimento de Jogos, Domain-Specific Language, Interactive Narratives, Compilers, Static Semantic Verification, State Mutation.

I. INTRODUÇÃO

O desenvolvimento de jogos do tipo RPG (Role-Playing Game) envolve a criação de narrativas interativas, personagens e sistemas de decisão que influenciam dinamicamente o fluxo da história. Esse tipo de aplicação exige a modelagem de estruturas complexas que combinam elementos narrativos e lógica de controle, frequentemente implementados por meio de

linguagens de programação de propósito geral, como Python ou C++.

Embora essas linguagens ofereçam flexibilidade, seu uso direto no desenvolvimento de jogos pode aumentar a complexidade da implementação, dificultar a manutenção e prejudicar a separação entre a especificação da narrativa e sua execução. Esse problema se torna mais evidente em sistemas baseados em eventos e escolhas, onde a lógica do jogo tende a se tornar fortemente acoplada à implementação.

Diante desse cenário, Linguagens Específicas de Domínio (DSLs) emergem como uma alternativa adequada, pois permitem a criação de abstrações mais próximas do domínio de jogos, facilitando a descrição de elementos narrativos de forma declarativa e estruturada. Trabalhos relacionados demonstram o uso de DSLs para representação de histórias interativas, geração de comportamentos e modelagem de jogos, evidenciando ganhos em produtividade e organização do código. No entanto, ainda existem limitações quanto à integração entre estrutura narrativa e mecanismos dinâmicos baseados em eventos de gameplay, especialmente em abordagens que não possuem uma definição formal clara da linguagem.

Considerando esse cenário, este trabalho tem como objetivo desenvolver uma DSL híbrida para a modelagem de jogos RPG baseados em narrativa textual, permitindo a definição de estruturas narrativas e, simultaneamente, a especificação de eventos, condições e ações de gameplay. A linguagem proposta busca equilibrar construções declarativas, voltadas à modelagem da narrativa, e construções imperativas, voltadas ao controle de execução e interação do jogo.

O escopo deste trabalho limita-se à definição de uma DSL textual para jogos RPG narrativos de estrutura linear e ramificada, contemplando a representação de aspectos declarativos da narrativa e mecanismos imperativos baseados em eventos de interação e tomada de decisão. Não faz parte do escopo a implementação de um motor gráfico completo ou a modelagem de física avançada, sendo o foco restrito à representação e execução da lógica narrativa.

Este artigo está organizado da seguinte forma: a Seção 2 apresenta os trabalhos relacionados no contexto de DSL aplicadas a jogos digitais; a Seção 3 descreve os materiais e métodos utilizados no desenvolvimento da DSL, incluindo sua especificação formal e modelo de execução; a Seção 4

apresenta os resultados obtidos e a análise da abordagem proposta; e, por fim, a Seção 5 apresenta as conclusões e perspectivas de trabalhos futuros.

II. TRABALHOS RELACIONADOS

O uso de Linguagens Específicas de Domínio (DSLs) no desenvolvimento de jogos digitais tem sido amplamente explorado como forma de aumentar a produtividade, modularidade e expressividade na modelagem de sistemas complexos.

Um dos trabalhos mais relevantes nesse contexto é o de Beyak e Carette [2] apresenta a DSL SAGA, voltada para o gerenciamento de narrativas em jogos. A abordagem modela histórias como sistemas de transição de estados, utilizando grafos acíclicos dirigidos e máquinas de estados para representar o fluxo narrativo. A linguagem prioriza a acessibilidade para designers não programadores, adotando uma sintaxe próxima da linguagem natural e permitindo a geração automática de código em linguagens orientadas a objetos. Embora eficiente na representação estrutural de narrativas, a proposta apresenta limitações na formalização explícita da linguagem e na integração com mecanismos dinâmicos de execução baseados em eventos.

No trabalho de Marques et al. [5], é proposta uma linguagem de modelagem específica de domínio (DSML) baseada em Desenvolvimento Dirigido por Modelos (MDD) para a criação de jogos RPG em dispositivos móveis. A abordagem utiliza metamodelos e ferramentas como o Eclipse Modeling Framework (EMF), um framework para definição e manipulação de modelos baseados em metadados, e a Atlas Transformation Language (ATL), uma linguagem voltada para transformações modelo-a-modelo, permitindo a geração automática de código a partir de modelos de alto nível. Um diferencial importante é o suporte à verificação formal por meio de model checking, garantindo propriedades lógicas do jogo ainda na fase de projeto. No entanto, a linguagem proposta é predominantemente visual e depende de um ecossistema específico de ferramentas, o que pode limitar sua flexibilidade e extensibilidade.

Marino et al. [4] exploram o uso de DSL no contexto de jogos de estratégia em tempo real (RTS), propondo a linguagem μ Language para a definição de estratégias programáticas. Nesse contexto, o sistema LS2 (Local Search and Language Simplifier) utiliza técnicas de busca local combinadas com mecanismos de simplificação de linguagem para gerar scripts de decisão eficientes e interpretáveis, onde a busca local é responsável por explorar e otimizar o espaço de possíveis estratégias, enquanto o processo de simplificação atua na redução da complexidade dos scripts gerados, eliminando redundâncias e preservando apenas decisões relevantes para o desempenho do agente. Embora o foco do trabalho esteja na síntese de estratégias em ambientes de jogos, a abordagem evidencia o potencial das DSLs na representação de comportamentos complexos e na separação entre especificação de alto nível e execução, aspectos também relevantes para a modelagem de sistemas baseados em eventos em jogos digitais.

Gill [3] propõe a DSL GypPO para a geração automática de jogos de plataforma 2D. A linguagem é estruturada em

componentes que separam a definição de elementos do jogo da lógica específica de cada nível, permitindo a geração de código em JavaScript a partir de uma representação intermediária baseada em AST. A abordagem demonstra a viabilidade de generalização de um gênero de jogos por meio de DSLs, embora não contemple explicitamente a modelagem de narrativas interativas ou eventos dinâmicos de gameplay.

De forma complementar, Alestig et al. [1] apresentam uma DSL embarcada para a descrição de jogos de tabuleiro, enfatizando a reutilização e a modularidade na definição de regras. Apesar de abordar um domínio distinto, o trabalho reforça a aplicabilidade de DSLs na modelagem de sistemas baseados em regras, ainda que com menor foco em execução dinâmica e eventos.

A análise dos trabalhos relacionados evidencia que, embora existam abordagens consolidadas para modelagem estrutural, geração de código e verificação formal em jogos, há uma lacuna na integração entre representação narrativa e mecanismos dinâmicos de execução, especialmente no contexto de jogos RPG baseados em eventos. Além disso, observa-se que muitas propostas não apresentam uma especificação formal completa da linguagem, o que limita sua capacidade de análise e extensibilidade.

TABLE I
COMPARAÇÃO ENTRE ABORDAGENS DE DSLs APLICADAS A JOGOS DIGITAIS

Trabalho	Narrativa	Eventos	AST	Formalização
DSL Board Games [1]	Média	Baixa	Não	Média
GypPO [3]	Média	Baixa	Sim	Média
LS2 / μ Language [4]	Baixa	Alta	Sim	Parcial
RPG DSL (MDD) [5]	Alta	Parcial	Não	Alta
SAGA [2]	Alta	Parcial	Não	Parcial

Diante dessas limitações, este trabalho propõe uma DSL híbrida que integra definições estruturais e comportamentais em um modelo unificado, com base em uma gramática formal e execução por meio de AST. A abordagem busca reduzir o acoplamento entre narrativa e lógica de controle, ao mesmo tempo em que permite a modelagem explícita de eventos de gameplay, posicionando-se como uma alternativa que combina expressividade, formalismo e suporte à execução dinâmica.

III. MATERIAL E MÉTODO

A. Especificação Formal da Linguagem

A especificação formal da linguagem proposta é definida por meio de uma gramática livre de contexto utilizando a notação EBNF (Extended Backus-Naur Form). Essa gramática estabelece a estrutura sintática da DSL, permitindo a descrição de elementos narrativos e eventos de gameplay de forma unificada.

Na Figura 1 são apresentados os elementos iniciais da gramática.

Essa gramática define os principais blocos estruturais da linguagem, incluindo a modelagem de cenas, personagens e eventos, bem como os mecanismos de controle de fluxo narrativo, como escolhas, transições e disparo de eventos.

A separação entre comandos narrativos e ações condicionais permite representar tanto aspectos declarativos quanto comportamentais do jogo.

A partir dessa definição, é possível realizar a análise sintática e gerar uma Árvore Sintática Abstrata (AST), que será posteriormente interpretada em ambiente Python para execução da lógica do jogo.

IV. RESULTADOS E DISCUSSÕES

Os resultados obtidos a partir do desenvolvimento e da consolidação da versão V4 evidenciam uma evolução significativa e multifacetada da DSL proposta. Sob a perspectiva técnica, estrutural e narrativa, a progressão sequencial entre as versões preliminares e o estágio atual demonstra um processo rigoroso e contínuo de refinamento de engenharia de software. Cada iteração metodológica contribuiu diretamente para que o sistema deixasse de ser um mero interpretador linear e se consolidasse como uma engine narrativa interativa funcional, robusta e computacionalmente segura. Esta solução atual é plenamente capaz de processar, validar estaticamente e executar jogos textuais complexos cujos desdobramentos dependem intimamente do gerenciamento de variáveis de estado dinâmicas. A análise comparativa aprofundada entre as versões permite compreender não apenas a expansão quantitativa das funcionalidades de gameplay implementadas, mas também o amadurecimento qualitativo da arquitetura de compiladores adotada.

A primeira versão da DSL (V1) representou o marco inicial do projeto, funcionando estritamente como uma etapa de validação de conceito. Nesse estágio embrionário, o principal objetivo científico consistia em verificar a viabilidade prática da criação de uma linguagem específica de domínio voltada para o contexto restrito de jogos narrativos baseados em escolha. A implementação da V1 foi construída sobre mecanismos elementares de análise léxica e sintática desenvolvidos de forma manual, sem o auxílio de geradores automáticos de parsers. Essa infraestrutura inicial permitia apenas a interpretação de estruturas primitivas da linguagem, tais como a delimitação de blocos de cena (*scene*), a exibição de diálogos estáticos na tela (*say*) e transições diretas e incondicionais entre os estados narrativos do grafo (*goto*). Apesar de severamente limitada em escopo e desprovida de lógica computacional interna, a V1 cumpriu o papel de estabelecer os fundamentos da sintaxe abstrata e definir os requisitos preliminares da engine de execução.

Entretanto, as avaliações práticas da primeira versão expuseram limitações severas no que tange à modularidade, escalabilidade e complexidade lógica. O parser construído manualmente gerava um código com alto acoplamento, o que dificultava a manutenção corretiva e inviabilizava a expansão da gramática livre de contexto para comportar novas regras de produção. Elementos semânticos cruciais para a caracterização de jogos de interpretação de papéis (RPGs), como o gerenciamento dinâmico de atributos numéricos, o disparo de eventos assíncronos, o desvio condicional de fluxo e o rastreamento de inventários, eram inexistentes nessa fase. Como consequência

direta, a experiência narrativa disponível na V1 reduzia-se a uma estrutura estritamente linear, configurando sequências previsíveis de cenas sem profundidade tática ou agência real por parte do usuário. Contudo, os dados brutos colhidos nessa fase inicial serviram como fundação acadêmica, comprovando que a estrutura de dados base da DSL possuía a integridade necessária para evoluir nas iterações subsequentes.

A segunda versão (V2) marcou uma quebra profunda de paradigma na arquitetura de software ao introduzir a biblioteca PLY (*Python Lex-Yacc*) como o núcleo do front-end do compilador. Essa transição tecnológica trouxe ganhos substanciais na organização matemática, na robustez algorítmica e na extensibilidade gramatical da linguagem. A disposição formal permitiu a separação estrita e modular entre o analisador léxico (*lexer*) e o analisador sintático (*parser*), alinhando o projeto com os padrões clássicos estabelecidos na teoria dos compiladores. Com isso, os padrões de busca e correspondência baseados em expressões regulares passaram a ser processados de forma centralizada e otimizada, refletindo-se diretamente em uma melhora drástica na legibilidade das regras de produção gramatical e na estabilidade do fluxo de derivação da DSL.

Ao longo do desenvolvimento da V2, a categorização dos tokens, o mapeamento de strings para palavras reservadas e a hierarquia de precedência das regras sintáticas ganharam consistência formal. Foi nesse período que a Árvore Sintática Abstrata (*Abstract Syntax Tree – AST*) passou a ser gerada como uma estrutura de dados orientada a objetos explícita, fornecendo uma representação computacional limpa e fiel das construções sintáticas da linguagem. Embora a engine da V2 ainda demonstrasse fragilidades funcionais no tocante à execução dinâmica de eventos complexos e na persistência reativa do estado global do jogo, essa iteração consolidou com sucesso a infraestrutura de engenharia necessária para as expansões futuras. Sob a ótica acadêmica, a incorporação do PLY validou a correlação prática do projeto com os conceitos teóricos ministrados em disciplinas de linguagens formais, autômatos e síntese de tradutores.

Não obstante o avanço estrutural consolidado pela V2, o produto final oferecido ao usuário final sob a forma de gameplay ainda mantinha características rudimentares. A linguagem oferecia apenas um suporte parcial para blocos isolados de diálogo, falhando em prover sistemas ramificados de tomada de decisão ou rotinas automatizadas de cálculo matemático sobre o inventário. Diante disso, a segunda versão deve ser caracterizada historicamente como um estágio intermediário de consolidação de infraestrutura, cujo propósito científico não era a entrega de recursos de jogabilidade avançados, mas sim o fortalecimento e a blindagem da arquitetura de parsing antes do acoplamento de mecânicas dinâmicas de maior complexidade algorítmica.

A terceira iteração da linguagem, denominada V3, elevou o patamar do projeto ao transformá-lo efetivamente em um motor de execução narrativa em tempo real. Nessa etapa, a capacidade expressiva da gramática foi expandida para suportar a declaração explícita de blocos de personagem (*character*),

a verificação rudimentar de expressões lógicas e um sistema primitivo de escolhas com ramificações permanentes no fluxo do jogo. A evolução do design narrativo foi evidente: o grafo linear e estrito das versões anteriores deu lugar a caminhos ramificados que começavam a simular a tomada de decisão de um RPG clássico. Contudo, a manipulação de regras de estado em tempo de execução e a garantia absoluta de integridade contra falhas nos scripts textuais ainda careciam de rigor formal, necessitando de uma camada adicional de maturidade técnica e validação matemática preventiva.

A culminação desse processo deu-se na quarta versão (V4), que reestruturou e solidificou em definitivo o projeto como uma linguagem de programação de domínio completo, estável e computacionalmente segura. Uma das principais transformações introduzidas na V4 foi a profunda modificação conceitual e sintática aplicada aos comandos de manipulação e mutação de estados aritméticos. A sintaxe foi expandida e vinculada a nós dedicados da AST, formalizados como os objetos *SetNode* (atribuição direta e tipada de valores literais), *AddNode* (incremento aritmético restrito) e *RemoveNode* (decremento aritmético restrito). Essa alteração permitiu que a engine realizasse alterações dinâmicas e seguras nos atributos quantitativos armazenados na tabela de símbolos global (como pontos de vida, reservas de energia, valores monetários e itens acumulados), provendo a base lógica necessária para simular sistemas de progressão numérica típicos de jogos complexos.

Adicionalmente, a expressividade lógica da versão V4 atingiu a maturidade ideal ao introduzir de forma nativa e robusta blocos de controle condicional estruturados sob a forma *if <expressao_logica> then <comando>* no escopo operacional das declarações de eventos (*event*). O parser da linguagem passou a aceitar e processar tokens baseados em operadores lógicos e relacionais de comparação com regras semânticas estritas, mapeando as operações de igualdade (*==*), desigualdade (*!=*), maior (*>*), menor (*<*), maior ou igual (*>=*) e menor ou igual (*<=*). Como resultado prático direto, as estruturas de eventos deixaram de agir como meros blocos estáticos de exibição textual e passaram a operar como componentes lógicos altamente reativos. A partir dessa implementação, a exibição de mensagens narrativas e a aplicação de consequências numéricas ao estado do personagem passaram a depender de asserções lógicas computadas em tempo de execução com base no histórico de escolhas anteriores do jogador.

Outra melhoria de extrema relevância para a usabilidade e engenharia da linguagem na versão V4 foi o suporte à inserção de comentários no código-fonte. O analisador léxico foi atualizado para ignorar o conteúdo textual precedido pelo token de barras duplas (*//*), canalizando esses blocos de texto para fora do pipeline de parsing. Essa funcionalidade, embora conceitualmente simples, alterou positivamente a experiência de uso da DSL, permitindo que escritores de conteúdo e designers de missões documentassem extensivamente a lógica interna dos grafos de cena, os pré-requisitos de eventos e o comportamento das variáveis globais sem causar erros sintáticos ou poluir o fluxo de processamento de tokens da

linguagem.

No campo da estabilidade estrutural e segurança de execução, a V4 trouxe uma das maiores contribuições científicas do projeto: a implementação de uma camada preventiva e explícita de verificação semântica (*semantic_check*). Esse componente atua de forma estática imediatamente após o parser concluir a montagem da AST e antes que qualquer instrução seja enviada ao interpretador runtime. O analisador semântico realiza uma varredura profunda e preditiva na árvore, mapeando erros lógicos e emitindo relatórios descritivos estruturados em três grandes áreas operacionais:

- 1) **Unicidade e Escopo de Identificadores:** Varre a tabela de símbolos para garantir a impossibilidade de declaração de cenas (*scene*) ou eventos (*event*) com identificadores duplicados, coibindo também a colisão de nomes na inicialização de variáveis de atributos do personagem.
- 2) **Checagem Estática de Tipos (*Type Checking*):** Garante a imutabilidade do tipo primitivo definido no momento da declaração das variáveis. O sistema intercepta e impede em tempo de compilação tentativas inválidas de atribuição (via *set*) de valores incompatíveis, além de barrar o uso das operações aritméticas de mutação (*add* e *remove*) quando aplicadas erroneamente sobre strings ou valores booleanos.
- 3) **Rastreabilidade e Validação do Grafo de Fluxo:** Executa uma busca em profundidade preditiva na estrutura narrativa para certificar que todos os destinos declarados em comandos *goto* e todas as ramificações mapeadas em blocos de escolha (*choice*) correspondam a identificadores de cenas que efetivamente existam no arquivo script de entrada, eliminando de forma definitiva falhas catastróficas por referências nulas ou caminhos sem saída.

Complementarmente à blindagem estática promovida pelo módulo semântico, a resiliência dinâmica do runtime na V4 foi aperfeiçoada por meio do monitoramento de estados críticos em tempo de execução com a criação da *Condição de Game Over*. O interpretador foi dotado de uma rotina de monitoramento que avalia o estado das variáveis após o disparo de qualquer mutação lógica. Caso atributos vitais pré-definidos (como os pontos de vida ou o nível de energia do herói) atinjam valores menores ou iguais a zero (≤ 0), a engine intercepta o estado, interrompe imediatamente o loop principal de entrada de comandos, exibe uma tela com mensagens personalizadas de encerramento da história e bloqueia de forma segura a inserção de novos dados pelo console, protegendo a consistência final da aplicação.

Demonstrando rigor nos critérios de engenharia de software, o ambiente de desenvolvimento do projeto na V4 passou por uma reestruturação física e didática completa. As células de execução do notebook foram segregadas e organizadas para refletir com exatidão as divisões clássicas de um compilador comercial de propósito geral. O código encontra-se dividido em seções modulares e estanques: Especificação

Léxica (Tokens), Análise Sintática (Regras de Produção Yacc), Definição dos Nós da AST, Analisador Semântico Preventivo e o Motor de Execução Interativo (*Runtime*). Essa organização não apenas facilitou a depuração de erros e a expansão do código do interpretador, mas também serviu como um sólido elemento de validação acadêmica, comprovando a viabilidade de aplicar os pilares teóricos da ciência da computação de forma organizada dentro de um ambiente notebook interativo.

Para validar de ponta a ponta as implementações incorporadas na versão V4, foi concebido e compilado o jogo experimental em larga escala intitulado "*Horizonte Rubro e o Mar Invertido*". O roteiro do teste foi desenhado intencionalmente para explorar exaustivamente todas as novas produções gramaticais e restrições lógicas da linguagem. O script utilizou comentários (//) para estruturar as fases, acionou eventos dinâmicos como *batalha_corsarios* para checar condições relacionais complexas ($vida > 30$), alterou o fluxo por meio de mutações simultâneas de estado (*add coragem 15* e *remove vida 25*) e manipulou bandeiras de progresso de itens exclusivos através da atribuição direta (*set mapa = 1*). Durante as sessões interativas de testes, o runtime exibiu um comportamento determinístico exemplar, respondendo corretamente aos comandos internos do console, efetuando rotinas automatizadas de limpeza de tela, computando o inventário em tempo real e disparando com precisão o encerramento do processo assim que as condições de falha eram atingidas. Sob a perspectiva acadêmica e científica, a V4 cumpriu com distinção todos os requisitos de design de linguagens, unificando análise formal, validação estática rigorosa e execução ágil em uma arquitetura de software limpa, desacoplada e performática.

V. CONCLUSÃO

O desenvolvimento do presente projeto de engenharia de software possibilitou a concepção, implementação e validação de uma Linguagem Específica de Domínio (DSL) robusta voltada para a criação de jogos narrativos interativos baseados em texto. O trabalho demonstrou com sucesso a viabilidade da transposição prática de conceitos densos e formais de engenharia de compiladores, análise semântica preventiva e gestão dinâmica de tabelas de símbolos para o cenário lúdico e comercial de jogos eletrônicos interpretados. Ao longo de todo o ciclo de desenvolvimento, observou-se uma evolução arquitetural nítida, na qual rotinas manuais isoladas e frágeis deram lugar a uma plataforma integrada, estável, modular e de altíssima capacidade expressiva na iteração final V4.

A trajetória evolutiva observada entre as versões V1, V2, V3 e V4 funcionou como uma sólida evidência do amadurecimento metodológico do projeto. A versão inicial V1 atuou de forma satisfatória como uma prova de conceito básica, estabelecendo contornos gramaticais primitivos com o uso de um analisador manual de strings. A iteração V2 reestruturou profundamente o front-end ao adotar a biblioteca PLY (*Python Lex-Yacc*), conferindo fundamentação matemática e elegância à extração de tokens e ao processamento sintático LALR(1) através da montagem automatizada de uma AST modular. A versão V3 transformou a DSL em uma linguagem

funcional capaz de guiar o usuário por caminhos de decisão múltiplos e permanentes. Finalmente, a versão definitiva V4 refinou e expandiu os comandos aritméticos essenciais (*set*, *add*, *remove*), incluiu o suporte formal a comentários de documentação de roteiro, acoplou controle condicional lógico-relacional dinâmico (*if-then*) e inseriu resiliência sistêmica ativa por meio da criação da lógica automatizada de *Game Over Condicional*.

Os resultados experimentais colhidos na iteração V4 atestam que a DSL cumpriu integralmente todos os objetivos científicos originalmente propostos. A linguagem demonstrou ser perfeitamente capaz de abstrair a complexidade sintática intrínseca das linguagens de propósito geral de alto nível, convertendo-as em primitivas textuais altamente legíveis e intuitivas para criadores de conteúdo, tais como *scene*, *choice*, *event* e *trigger*. O grande trunfo de engenharia da versão final concentrou-se no desenvolvimento do analisador semântico estático aliado à distribuição didática das fases do compilador no ambiente de desenvolvimento. Ao realizar a varredura prévia de tipos, mitigar a colisão de nomes e prever a existência de grafos órfãos no fluxo de cenas, a DSL garantiu um nível de segurança operacional comparável ao de compiladores comerciais, eliminando falhas fatais em tempo de execução decorrentes de erros humanos de digitação nos roteiros.

Sob a perspectiva acadêmica, o projeto obteve êxito total ao unificar tópicos complexos de ciência da computação — como gerenciamento de memória em tabelas de símbolos, grafos de transição dirigidos e derivação sintática abstrata — a um domínio prático de desenvolvimento de jogos digitais. A arquitetura desacoplada implementada em Python comprovou que a segregação estrita entre a camada declarativa da história (cenas, escolhas e diálogos) e a especificação das mecânicas lógicas subjacentes (eventos, condicionais e mutações numéricas) reduz drasticamente o acoplamento do código, tornando-o legível e amigável tanto para engenheiros de software quanto para designers de narrativas e escritores.

Considerando o elevado patamar de estabilidade e maturidade técnica alcançado pela versão V4, o projeto consolida uma infraestrutura sólida e fértil para desdobramentos e investigações futuras no campo de linguagens de programação. Como linhas de pesquisa subsequentes, aponta-se o desenvolvimento de suporte a escopos locais de variáveis (permitindo atributos exclusivos e isolados para personagens não-jogáveis ou salas específicas), a inclusão de analisadores de expressões matemáticas complexas com parênteses nas rotinas de atribuição, o desenvolvimento de um subsistema de persistência de dados para suporte nativo a *save/load game* e a integração com modelos de processamento de linguagem natural (PLN) para recepção de respostas abertas via console. Adicionalmente, vislumbra-se a criação de um ambiente integrado de desenvolvimento visual (IDE) baseado em interfaces de nós interconectados, o que permitirá que criadores com nenhuma experiência prévia em computação utilizem todo o poder de compilação da DSL para dar vida a narrativas complexas, imersivas e ricas em mecânicas lógicas.

REFERÊNCIAS

- [1] E. Alestig *et al.*, “A Language for Board Games: Development of an Embedded Domain-Specific Language for Describing Board Games,” B.S. thesis, Chalmers University of Technology and University of Gothenburg, Gothenburg, Sweden, 2022.
- [2] L. Beyak and J. Carette, “SAGA: A DSL for Story Management,” Department of Computing and Software, McMaster University, Hamilton, ON, Canada, 2014.
- [3] P. Gill, “GypPO: A DSL and Code Generator for Platformer Games,” M.S. thesis, McMaster University, Hamilton, ON, Canada, 2018.
- [4] J. R. H. Marino, R. O. Moraes, T. C. Oliveira, C. Toledo, and L. H. S. Lelis, “Programmatic Strategies for Real-Time Strategy Games,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.
- [5] E. Marques, V. Balegas, B. F. Barroca, A. Barisic, and V. Amaral, “The RPG DSL: A Case Study of Language Engineering Using MDD for Generating RPG Games for Mobile Phones,” in *Proceedings of the 12th Workshop on Domain-Specific Modeling (DSM)*, Tucson, AZ, USA, 2012.
- [6] M. Preuss *et al.*, “LS2: Learning to Design Game Strategies Using Domain-Specific Languages,” in *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, 2018.
- [7] AshesBorn, “DSL-RPG,” GitHub repository, Available: <https://github.com/AshesBorn/DSL-RPG>. Accessed on: May. 25, 2026.

```

<programa>      ::= { <declaracao> }

<declaracao>    ::= <cena> | <personagem> | <evento>

<cena>          ::= "scene" <identificador> "{" { <comando> } }"

<personagem>    ::= "character" <identificador> "{" { <atributo> } }"

<evento>        ::= "event" <identificador> "{" { <acao> } }"

<comando>       ::= <dialogo> | <escolha> | <transicao> | <disparo_evento>

<dialogo>       ::= "say" <string> ";"

<escolha>       ::= "choice" "{" { <opcao> } }"

<opcao>         ::= <string> "->" <identificador> ";"

<transicao>      ::= "goto" <identificador> ";"

<disparo_evento> ::= "trigger" <identificador> ";"

<atributo>      ::= <identificador> "=" <valor> ";"

<acao>          ::= "if" <condicao> "then" <comando>

<condicao>       ::= <identificador> <operador> <valor>

<operador>      ::= "==" | "!=" | ">" | "<" | ">=" | "<="

<valor>         ::= <numero> | <string> | <booleano>

<identificador> ::= letra { letra | digito }

<string>        ::= '"' { caractere } '"'

<numero>        ::= digito { digito }

<booleano>      ::= "true" | "false"

```

Fig. 1. Gramática EBNF da DSL na fase inicial

```

<programa> ::= { <declaracao> }

<declaracao> ::= <cena> | <personagem> | <evento>

<cena> ::= "scene" <identificador> "{" { <comando> } "}"

<personagem> ::= "character" <identificador> "{" { <atributo> } "}"

<evento> ::= "event" <identificador> "{" { <acao> } "}"

<comando> ::= <dialogo> | <escolha> | <transicao> | <disparo_evento>

<dialogo> ::= "say" <string> ";"

<escolha> ::= "choice" "{" { <opcao> } "}"

<opcao> ::= <string> "->" <identificador> ";"

<transicao> ::= "goto" <identificador> ";"

<disparo_evento> ::= "trigger" <identificador> ";"

<atributo> ::= <identificador> "=" <valor> ";"

<acao> ::= <mutacao> | <condicional> | <dialogo>

<mutacao> ::= <op_mutacao> <identificador> <valor> ";"

<op_mutacao> ::= "set" | "add" | "remove"

<condicional> ::= "if" <condicao> "then" <acao>

<condicao> ::= <identificador> <operador> <valor>

<operador> ::= "==" | "!=" | ">" | "<" | ">=" | "<="

<valor> ::= <numero> | <string> | <booleano>

<identificador> ::= ( letra | "_" ) { letra | digito | "_" }

<string> ::= '"' { caractere } '"'

<numero> ::= digito { digito }

<booleano> ::= "true" | "false"

<comentario> ::= "//" { caractere }

```

Fig. 2. Gramática EBNF formal da DSL na versão V4.